



SecureAccess – Personal File Vault

Abstract

SecureAccess is a desktop-based personal file vault application developed using Python to ensure secure storage and protection of sensitive user files. The system integrates user authentication with strong cryptographic mechanisms to provide confidentiality, integrity, and controlled access to data. By leveraging password-based key derivation and authenticated encryption, SecureAccess mitigates common security threats such as unauthorized access, brute-force attacks, and file tampering. This project demonstrates the practical application of cybersecurity principles in a real-world desktop software environment.

1. Introduction

In the modern digital era, individuals increasingly store personal and sensitive information on their computers, making file security a critical concern. Unauthorized access, malware, and physical theft of storage devices pose significant risks to personal data. Traditional file hiding or basic password protection mechanisms are often insufficient against determined attackers.

SecureAccess addresses these challenges by providing a robust personal file vault that encrypts files using industry-standard cryptographic techniques. The application is designed as a desktop solution with a graphical user interface, making it accessible to non-technical users while maintaining strong security guarantees. The project focuses on implementing secure authentication, encryption, and access control using Python.

2. System Architecture

SecureAccess follows a modular system architecture that separates concerns between the user interface, core security logic, and data storage. The architecture consists of the following components:

2.1 User Interface Layer

The user interface is built using CustomTkinter, providing a modern and intuitive GUI. It allows users to create accounts, log in securely, and perform file encryption and decryption operations. The UI acts as the interaction layer and does not directly handle sensitive cryptographic operations.

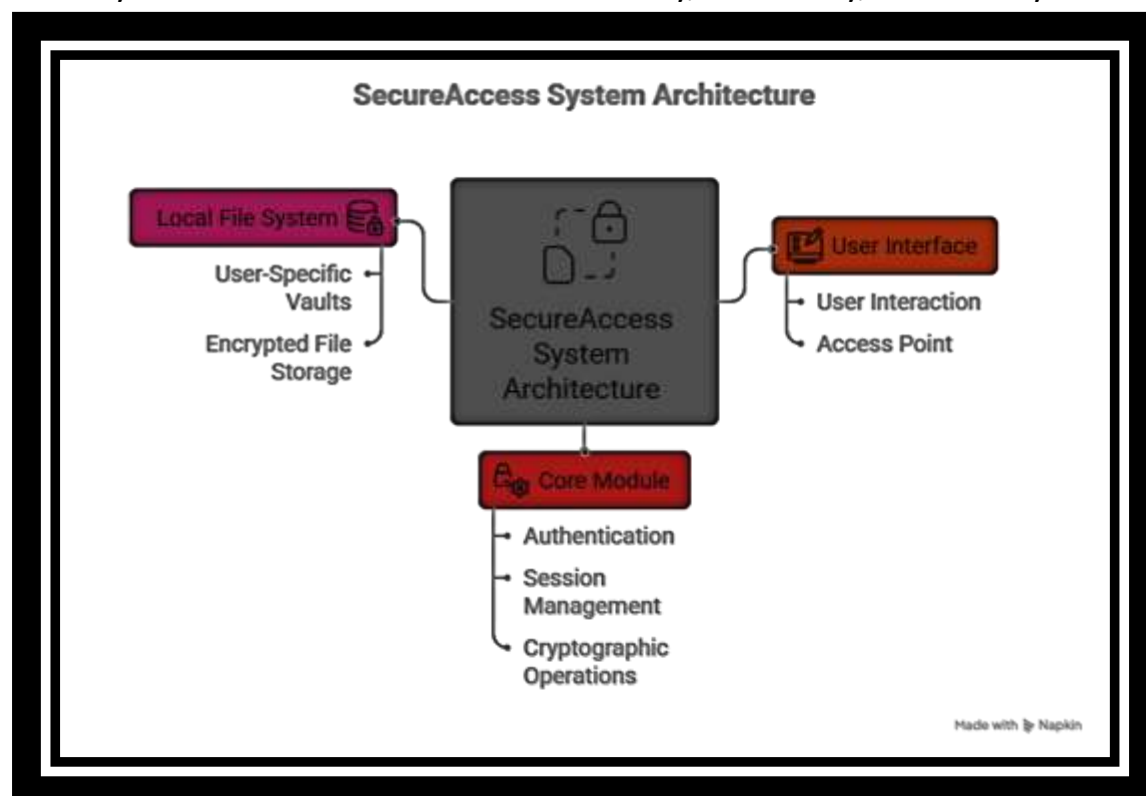
2.2 Core Security Module

The core module handles authentication, session management, and cryptographic processes. User passwords are hashed before storage, and encryption keys are derived dynamically when required. This module ensures that security logic is isolated from the interface, improving maintainability and reducing attack surface.

2.3 Storage Layer

The storage layer consists of the local file system. Encrypted files are stored in user-specific vault directories. Sensitive runtime data such as user credentials and encrypted files are excluded from version control to prevent accidental exposure.

This layered architecture enhances security, scalability, and clarity of design.



3. Encryption and Decryption Process

3.1 Key Derivation

SecureAccess does not use user passcodes directly as encryption keys. Instead, it applies the PBKDF2 (Password-Based Key Derivation Function 2) algorithm with the SHA-256 hashing function. A random salt and a high iteration count are used to derive a strong cryptographic key. This approach significantly increases resistance to brute-force and dictionary attacks.

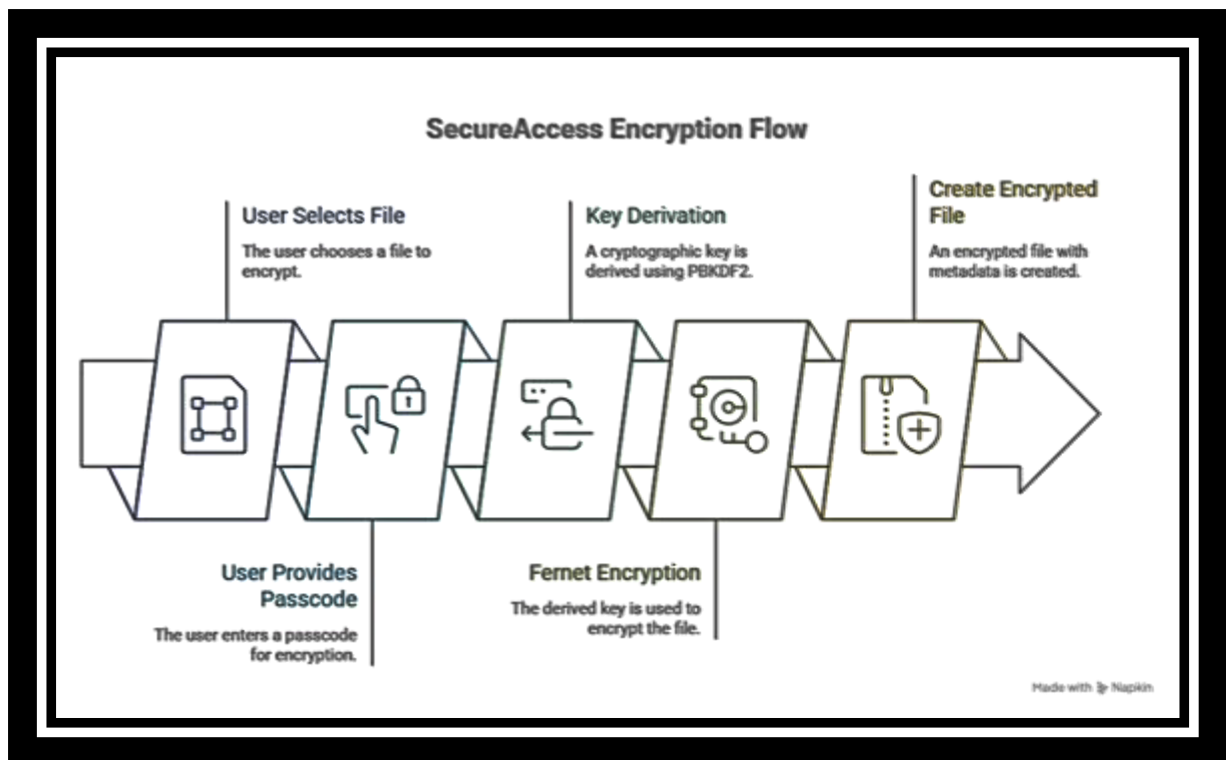
3.2 Encryption Flow

When a user selects a file for encryption, the system performs the following steps:

1. The user provides a passcode.
2. A random salt is generated.
3. PBKDF2 derives a secure key using the passcode, salt, and iteration count.
4. The file is encrypted using the Fernet scheme, which internally applies AES encryption with integrity verification.
5. A custom file header containing metadata (magic bytes, version, salt, iteration count) is prepended to the encrypted file.

3.3 Decryption Flow

For decryption, the system reads the file header to extract cryptographic parameters. The user supplies the passcode, and the same key derivation process is repeated. If the passcode is correct and the file has not been tampered with, the original file content is restored. Incorrect passcodes or modified files result in decryption failure, preserving security.



4. Threat Model and Security Analysis

SecureAccess is designed to defend against several common threat scenarios:

4.1 Unauthorized File Access

An attacker may gain access to encrypted files through theft or unauthorized copying. Since files are encrypted using strong cryptography, the attacker cannot read the contents without the correct passcode.

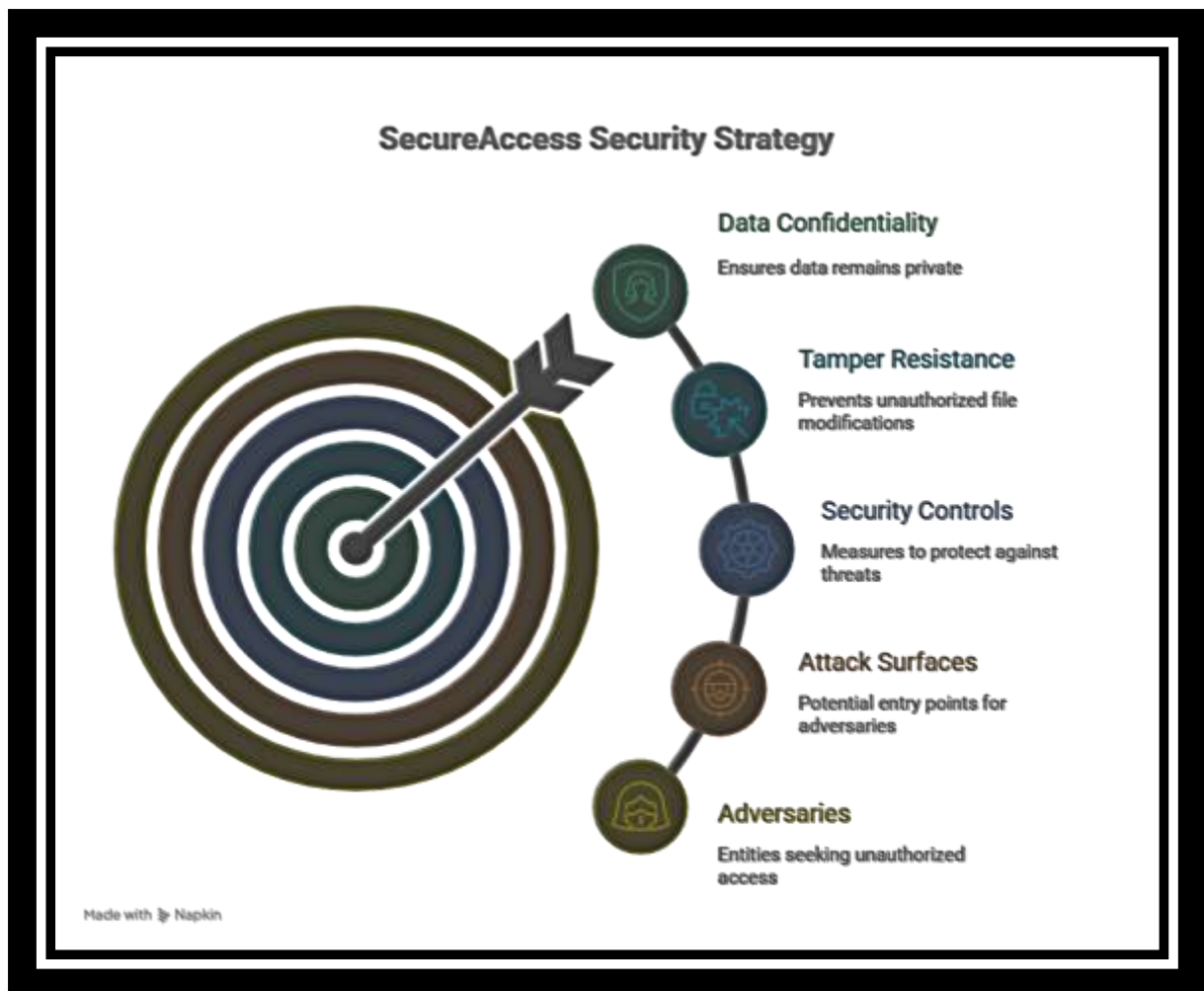
4.2 Brute-Force Attacks

To counter brute-force attempts, SecureAccess uses PBKDF2 with a high iteration count. This significantly slows down each password attempt, making large-scale guessing attacks computationally expensive.

4.3 File Tampering

Authenticated encryption ensures that any modification to encrypted files is detected during decryption. Tampered files are rejected, preventing corrupted or malicious data from being processed.

The combination of these controls ensures a strong security posture suitable for personal data protection.



5. Implementation Details

- **Programming Language:** Python
- **GUI Framework:** CustomTkinter
- **Cryptography Library:** Cryptography (Fernet, PBKDF2)
- **Password Hashing:** SHA-256
- **Storage:** Local file system with ignored sensitive files

The application is designed to be platform-independent and lightweight, making it suitable for personal computers.

6. Applications and Use Cases

- Personal file protection
 - Academic cybersecurity projects
 - Demonstration of applied cryptography
 - Desktop security tool prototype
-

7. Conclusion

SecureAccess successfully demonstrates how strong cryptographic techniques and secure software design principles can be applied to protect personal files. By combining user authentication, password-based key derivation, and authenticated encryption within a clean desktop interface, the system provides an effective solution for secure file storage. The project highlights the importance of proper key management and threat modeling in modern software development and serves as a solid foundation for further enhancements such as cloud integration or multi-factor authentication.

References

1. NIST SP 800-132: Recommendation for Password-Based Key Derivation
2. Python Cryptography Documentation
3. OWASP Secure Storage Guidelines